

Similarity Search Doc 2

This is a document outlining progress on the AI front, at which there has been substantial progress with upgrading our similarity search demo to account for our Supabase database, as well as revised similarity metrics.

Chosen Similarity Metric

$$d(a, b) = d(b, a) = \sqrt{\sum_{i=0}^{n-1} (b_i - a_i)^2}$$

where $a = (a_0, a_1, \dots, a_{n-1})$ and $b = (b_0, b_1, \dots, b_{n-1})$ are two points in n -dimensional Euclidean space

To replicate the most accurate similarity metric to one that would resemble real life similarity, all while keeping the scope feasible, I suggest we use the *Euclidean distance* similarity metric.

- Essentially, Euclidean distance measures the length of a segment that connects 2 points.

This suits our use case well as it provides a similar metric to what someone in real-life would call a similar start location.

- For example, if two employee's live 2 minutes walk away from one another, the distance between them will be small, hence the start location is very 'similar'.

Database Connectivity

For our project, we have chosen Supabase to host our data storage on their free plan.

This enables us to access their cloud-based database through PostgreSQL, while maintaining data coherency between our sub-teams.

The database is very easy to connect to, this can be done by doing the steps described [here](#).

Updated Similarity Search Walkthrough

To avoid having to insert, drop, and reinsert data like in our previous demo, we have optimised the process to insert the correct data into the Milvus collection once, then we can run the demo many times much quicker.

A first time run would require running the script ('insert-milvus-collection.py') to insert the collection of data into Milvus.

Then, when ready, we can run the script ('similarity-search-1.2.py') to run the similarity search.

1. Connects to our database.
2. Connects to Milvus.
3. Defines the search parameters
4. Performs the search on our chosen employee.
5. Prints users with similar journeys.

Remember, the collection will remain inside Docker until we run the ('drop-milvus-collections.py') script.